Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

Рубежный контроль № 1

Вариант А-1

Дисциплина: «Парадигмы и конструкции языков программирования»

Выполнил: Студент ИУ5-32Б (группа) _____ (подпись, дата) А.А.Абаньшина (И.О. Фамилия)

Проверил: _____ (подпись, дата) Ю.Е. Гапанюк (И.О. Фамилия)

Дата сдачи(защиты):

Результаты сдачи (защиты):
-Балльная оценка:
- Оценка :

2026 г.

Описание задания

Разработать программу на языке Python, реализующую классы «Студент» и «Группа», связанные отношением один-ко-многим, а также дополнительный промежуточный класс для реализации связи многие-ко-многим.

А1. Вывести список всех связанных студентов и учебных групп, отсортированный по названиям групп; порядок сортировки студентов внутри групп произвольный.

А2. Вывести список групп с суммарным размером стипендии студентов в каждой группе, отсортированный по сумме стипендий. Для выполнения запроса у каждого студента предусмотрен количественный признак — размер стипендии.

А3. По связи многие-ко-многим вывести список всех групп, в названии которых присутствует слово «группа», и списки обучающихся в них студентов.

Для тестирования программы заданы пять студентов, четыре учебные группы и записи промежуточной связи многие-ко-многим. Идентификаторы объектов используются для связывания записей между собой.

```
from operator import itemgetter


class Schoolboy:
    def __init__(self, id, fio, grade, class_id):
        self.id = id
        self.fio = fio
        self.grade = grade
        self.class_id = class_id


class SchoolClass:
    def __init__(self, id, name):
        self.id = id
        self.name = name


class SchoolboyClass:
    def __init__(self, class_id, schoolboy_id):
        self.class_id = class_id
        self.schoolboy_id = schoolboy_id


classes = [
    SchoolClass(1, '5А класс'),
    SchoolClass(2, '6Б класс'),
    SchoolClass(3, '7В класс'),
    SchoolClass(11, '5А (другой) класс'),
    SchoolClass(22, '6Б (другой) класс'),
    SchoolClass(33, '(другой) 7В класс'),
]

schoolboys = [
    Schoolboy(1, 'Артамонов', 4.5, 1),
```

```
    Schoolboy(2, 'Петров', 3.8, 2),
    Schoolboy(3, 'Иваненко', 4.9, 3),
    Schoolboy(4, 'Иванов', 4.2, 3),
    Schoolboy(5, 'Иванин', 3.5, 3),
]

schoolboys_classes = [
    SchoolboyClass(1, 1),
    SchoolboyClass(2, 2),
    SchoolboyClass(3, 3),
    SchoolboyClass(3, 4),
    SchoolboyClass(3, 5),
    SchoolboyClass(11, 1),
    SchoolboyClass(22, 2),
    SchoolboyClass(33, 3),
    SchoolboyClass(33, 4),
    SchoolboyClass(33, 5),
]


def main():
    one_to_many = [(s.fio, s.grade, c.name)
                   for c in classes
                   for s in schoolboys
                   if s.class_id == c.id]

    many_to_many_temp = [(c.name, sc.class_id, sc.schoolboy_id)
                         for c in classes
                         for sc in schoolboys_classes
                         if c.id == sc.class_id]

    many_to_many = [(s.fio, s.grade, class_name)
                    for class_name, class_id, schoolboy_id in
many_to_many_temp
                    for s in schoolboys if s.id == schoolboy_id]

    # Задание A1
    print("Задание A1. Школьники и классы (по классам):")
    res_11 = sorted(one_to_many, key=lambda x: x[2].casefold())
    for fio, grade, class_name in res_11:
        print(f"{fio} (балл {grade}) - {class_name}")

    print()  # Пустая строка

    # Задание A2
    print("Задание A2. Суммарный балл школьников по классам (по убыванию):")
    res_12_unsorted = []
    for c in classes:
        c_schoolboys = list(filter(lambda i: i[2] == c.name, one_to_many))
        if len(c_schoolboys) > 0:
            c_grades = [grade for _, grade, _ in c_schoolboys]
            c_grades_sum = sum(c_grades)
            res_12_unsorted.append((c.name, c_grades_sum))
    res_12 = sorted(res_12_unsorted, key=lambda x: -x[1])
    for class_name, total in res_12:
        print(f"{class_name}: {total}")

    print()  # Пустая строка

    # Задание A3
    print("Задание A3. Классы со словом «класс» и их школьники (многие-ко-
многим):")
    res_13 = {}
    for c in classes:
```

```
        if 'класс' in c.name:
            c_schoolboys = list(filter(lambda i: i[2] == c.name,
many_to_many))
            c_schoolboys_names = sorted(x for x, _, _ in c_schoolboys)
            res_13[c.name] = c_schoolboys_names
    for class_name, names in res_13.items():
        print(f"{class_name}:")
        for name in names:
            print(f"    - {name}")


if __name__ == '__main__':
    main()
```

```
C:\Users\andre\PycharmProjects\lab1\.venv\Scripts\python.exe C:\Users\andr
Задание A1. Школьники и классы (по классам):
Артамонов (балл 4.5) - 5А класс
Петров (балл 3.8) - 6Б класс
Иваненко (балл 4.9) - 7В класс
Иванов (балл 4.2) - 7В класс
Иванин (балл 3.5) - 7В класс

Задание A2. Суммарный балл школьников по классам (по убыванию):
7В класс: 12.600000000000001
5А класс: 4.5
6Б класс: 3.8

Задание A3. Классы со словом «класс» и их школьники (многие-ко-многим):
5А класс:
    - Артамонов
6Б класс:
    - Петров
7В класс:
    - Иваненко
    - Иванин
    - Иванов
5А (другой) класс:
    - Артамонов
6Б (другой) класс:
    - Петров
(другой) 7В класс:
    - Иваненко
    - Иванин
    - Иванов
```

Первый и второй запросы используют связь один-ко-многим между студентом и учебной группой. Третий запрос использует связь многие-ко-многим через промежуточный класс StudentGroup. Для формирования результатов применены списковые включения, функции sorted и sum, а также проверка наличия подстроки с помощью оператора in.